

A
PROJECT-III REPORT
on
ResumeIQ: An AI-Powered Resume Analyzer

Submitted by:

Navya Pathak

(230758)

Daksh Dureja

(230579)

Harman Singh

(230656)

Jaspreet Kamboj

(230757)

under mentorship of

Dr. Satyendr Singh

(Assistant Professor)



Department of Computer Science and Engineering
School of Engineering and Technology
BML Munjal University

May 2026

CANDIDATE'S DECLARATION

I hereby certify that the work on the project entitled, "ResumeIQ: An AI-Powered Resume Analyzer – A Production-Grade SaaS Platform for Resume, LinkedIn, and GitHub Profile Analysis", in partial fulfilment of requirements for the award of Degree of Bachelor of Technology in School of Engineering and Technology at BML Munjal University, having University Roll No.1232434, is an authentic record of my own work carried out during the academic session under the supervision of Dr. Satyendr Singh

Navya Pathak (230758)

Daksh Dureja (230579)

Harman Singh (230656)

Jaspreet Kamboj (230757)

Date: 4th May 2026

SUPERVISOR'S DECLARATION

This is to certify that the above statement made by the candidate is correct to the best of my knowledge and the project work has been accomplished satisfactorily.

Faculty Supervisor Name: Dr. Satyendr Singh

Signature:

Date: 4th May 2026

ACKNOWLEDGEMENT

We are highly grateful to Dr. Satyendr Singh, Designation, BML Munjal University, Gurugram, for providing invaluable supervision and guidance during the course of this project. Their expert advice, timely feedback, and continuous encouragement have been instrumental in the successful completion of ResumeIQ.

We extend my sincere gratitude to the Department of Computer Science Engineering, School of Engineering and Technology, BML Munjal University, for providing the necessary resources, infrastructure, and academic environment conducive to research and development.

We would also like to acknowledge the developers and maintainers of open-source tools and technologies including Next.js, React, Tailwind CSS, shadcn/ui, Groq AI (Llama-3.3-70B), Google Generative AI, and pdf-parse-fork whose contributions made this project feasible and production-ready.

Finally, we are deeply thankful to our family, friends, and colleagues who provided moral support and constructive feedback throughout the development process.

Navya Pathak

Daksh Dureja

Harman Singh

Jaspreet Kamboj

Department of Computer Science Engineering

BML Munjal University, Gurugram

TABLE OF CONTENTS

Section	Page No.
Candidate's Declaration	ii
Supervisor's Declaration	ii
Acknowledgement	iii
Table of Contents	iv
List of Figures	v
List of Tables	v
List of Abbreviations	vi
Abstract	vii
Chapter 1: Introduction	1
1.1 Introduction to the Project	1
1.2 Motivation and Background	2
1.3 Problem Statement	3
1.4 Objectives of the Project	4
1.5 Scope of the Project	4
1.6 User Requirement Analysis	5
1.7 Feasibility Study	5
Chapter 2: Literature Review	6
2.1 Overview of ATS Systems	6
2.2 AI in Resume Screening	7
2.3 GitHub Profile Analytics	8
2.4 LinkedIn Profile Optimization	8
2.5 Comparison of Existing Systems	9
Chapter 3: Methodology	10
3.1 System Architecture Overview	10
3.2 Technology Stack	11
3.3 Frontend Design	12
3.4 Backend and API Design	13
3.5 AI Integration and Model Selection	14
3.6 Caching System Design	15
3.7 Authentication System	16
3.8 Database Design and File Structure	17

3.9 Use Case Diagram and Flowcharts	18
Chapter 4: Module-wise Implementation	19
4.1 ATS Resume Analyzer Module	19
4.2 GitHub Profile Analyzer Module	21
4.3 LinkedIn Profile Analyzer Module	23
4.4 Job Board Module	24
4.5 Authentication and User Profile Module	25
Chapter 5: Results and Discussion	26
5.1 ATS Score Analysis Results	26
5.2 GitHub Analysis Results	28
5.3 LinkedIn Analysis Results	29
5.4 Performance Analysis	30
5.5 System Testing	31
Chapter 6: Conclusion and Future Scope	33
6.1 Conclusion	33
6.2 Future Scope	34
References / Bibliography	36
List of Abbreviations	38

LIST OF FIGURES

Figure No.	Figure Description	Page No.
Fig 1.1	High-Level System Architecture of ResumeIQ	10
Fig 1.2	Landing Page / Hero Section of ResumeIQ	12
Fig 2.1	ATS Resume Analyzer – Upload Interface	19
Fig 2.2	ATS Score Results – Score Breakdown Dashboard	20
Fig 2.3	ATS Score – Keywords Section and Suggestions	21
Fig 3.1	GitHub Analyzer – Input and Results Interface	22
Fig 3.2	GitHub Analysis Sections – Repositories, Activity, Skills	23
Fig 4.1	LinkedIn Analyzer – Input and Output Interface	24
Fig 4.2	LinkedIn Section Score Breakdown	24
Fig 5.1	Job Board Page – Browse and Filter Jobs	25
Fig 5.2	Job Detail Page – Description and Skills	25
Fig 6.1	User Profile Page	26
Fig 6.2	Login / Signup Page Interface	26
Fig 7.1	ATS Caching System Flowchart	16
Fig 7.2	Authentication Flow Diagram	17

LIST OF TABLES

Table No.	Table Description	Page No.
Table 1.1	User Requirement Analysis Matrix	5
Table 2.1	Comparison of Existing Resume Analysis Systems	9
Table 3.1	Technology Stack Summary	11
Table 3.2	ATS Scoring Criteria Breakdown	15
Table 3.3	Database / Cache Entry Structure	17
Table 5.1	ATS Scoring Ranges and Interpretation	27
Table 5.2	System Performance Metrics	30
Table 5.3	Test Cases and Results	32

LIST OF ABBREVIATIONS

Abbreviation	Full Form
ATS	Applicant Tracking System
AI	Artificial Intelligence
API	Application Programming Interface
SaaS	Software as a Service
LLM	Large Language Model
PDF	Portable Document Format
JWT	JSON Web Token
SHA256	Secure Hash Algorithm 256
OCR	Optical Character Recognition
CSS	Cascading Style Sheets
JSX	JavaScript XML
TSX	TypeScript XML
UI	User Interface
UX	User Experience
HTTP	Hypertext Transfer Protocol
REST	Representational State Transfer
JSON	JavaScript Object Notation
CDN	Content Delivery Network
NLP	Natural Language Processing
GROQ	Groq AI Inference API
ENV	Environment Variable

ABSTRACT

ResumeIQ is a production-grade, AI-powered Software-as-a-Service (SaaS) platform designed to help job seekers optimize their professional profiles and improve their chances of success in the modern, digitally-driven recruitment landscape. The platform integrates three powerful analytical modules: an ATS (Applicant Tracking System) Resume Analyzer, a GitHub Profile Analyzer, and a LinkedIn Profile Analyzer.

In the contemporary recruitment ecosystem, a large percentage of resumes are automatically filtered by ATS software before they are reviewed by human recruiters. Candidates who do not optimize their resumes for ATS algorithms regardless of their qualifications are frequently eliminated from the application process. Similarly, technical roles now heavily scrutinize candidates' GitHub profiles, and professional networks like LinkedIn have become indispensable tools in talent acquisition. ResumeIQ addresses these challenges holistically by providing intelligent, AI-driven feedback across all three dimensions.

The ATS module accepts resumes in PDF or image format, extracts the text content using advanced parsing libraries, and then passes the extracted text to the Groq AI inference engine (powered by the Llama-3.3-70B language model) for a rigorous evaluation. The analysis produces a detailed score (out of 100) broken down into four sub-categories: Formatting (0–25 points), Keywords (0–35 points), Structure (0–20 points), and Content (0–20 points). Alongside the score, the system provides actionable suggestions, identified strengths and weaknesses, and lists of matched and missing keywords relative to the target job role. An intelligent SHA-256-based caching mechanism ensures that repeated analyses of the same resume and role combination return instant results without redundant API calls.

The GitHub Analyzer module fetches a candidate's publicly available GitHub data including profile details and up to ten most recently updated repositories through the GitHub REST API. This data is then analyzed by the Groq AI model to produce a score and detailed feedback across four dimensions: Repositories, Activity, Skills, and Documentation. The LinkedIn Analyzer accepts manual profile data input from the user and evaluates it across four professional dimensions: Headline, About, Experience, and Skills.

The platform is built using Next.js 16 with React 19 for the frontend, TypeScript for type safety, and Tailwind CSS with shadcn/ui for a polished, mobile-responsive user interface. Authentication is handled via NextAuth.js with support for both Google OAuth and credential-based login. A built-in Job Board module provides a curated, searchable, and filterable list of tech jobs with detailed descriptions and skill requirements. The system is designed for deployment on Vercel and is fully compatible with Docker-based containerization.

Keywords: ATS, Resume Analysis, AI, Groq, Llama-3, GitHub Analyzer, LinkedIn Optimizer, Next.js, SaaS, Job Board, Natural Language Processing.

CHAPTER 1: INTRODUCTION

1.1 Introduction to the Project

ResumeIQ is a comprehensive, production-grade SaaS (Software-as-a-Service) web application built to bridge the gap between job seekers and the modern recruitment pipeline. The platform empowers candidates with data-driven, AI-generated insights about their resumes, GitHub profiles, and LinkedIn profiles — the three most critical pillars of a developer's professional identity in the current job market.

The system is structured as a monorepo hosted in a single repository, with the primary application residing in the apps/web directory. It is developed using Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS v4, and shadcn/ui component library. The backend logic is implemented as Next.js API routes, leveraging the Groq AI API for language model inference, the Google Gemini SDK (installed but serving as a fallback), the GitHub REST API for public data, pdf-parse-fork for server-side PDF text extraction, and NextAuth.js for authentication.

The platform is organized into five major modules:

- **ATS Resume Analyzer:** Accepts PDF/image resumes and generates a comprehensive ATS compatibility score with detailed sub-category breakdown and actionable suggestions.
- **GitHub Profile Analyzer:** Fetches public GitHub data and generates a profile strength score with feedback on repositories, activity, skills, and documentation quality.
- **LinkedIn Profile Analyzer:** Accepts manually inputted LinkedIn profile text and evaluates it against the target job role across key profile dimensions.
- **Job Board:** Provides a searchable and filterable list of curated tech job listings with detailed job descriptions, requirements, and skill tags.
- **Authentication & User Profile:** Supports Google OAuth and email/password login, with a personal profile page for authenticated users.

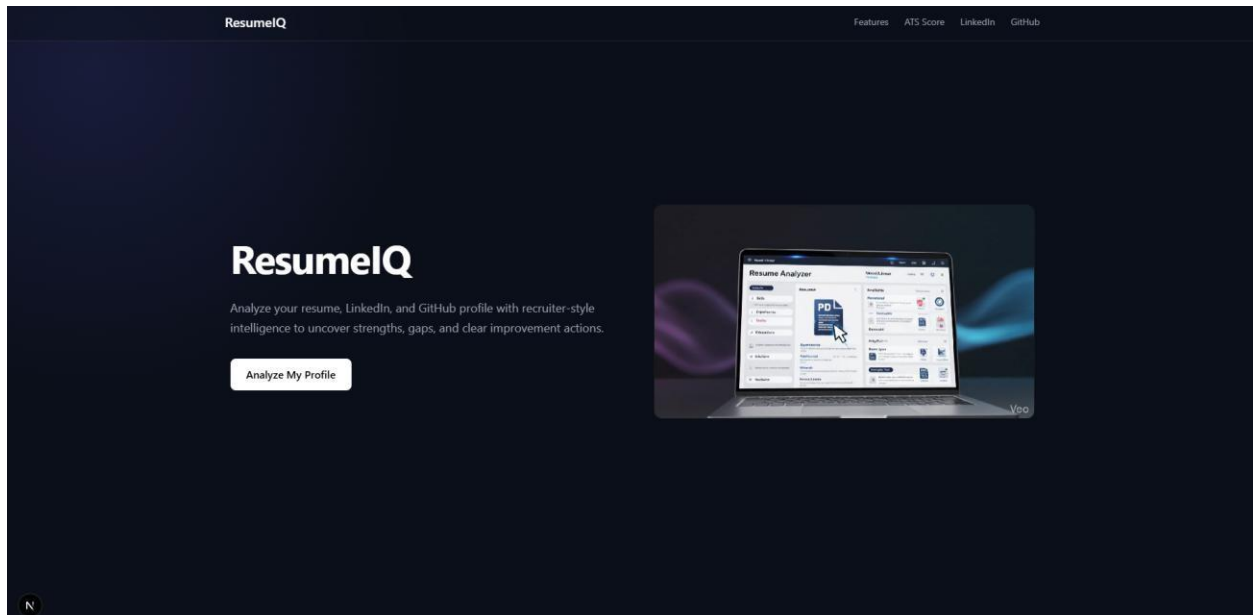


Figure 1.1: Landing Page / Hero Section of ResumeIQ

1.2 Motivation and Background

The recruitment landscape has undergone a dramatic transformation in the past decade. Large organizations receive thousands of applications for a single job posting, making it virtually impossible for human recruiters to manually screen every resume. To address this volume, companies have widely adopted Applicant Tracking Systems (ATS) — software platforms that automatically parse, filter, and rank resumes based on keyword matching, formatting compliance, and structural completeness. Studies suggest that over 75% of resumes are rejected by ATS before they ever reach a human recruiter.

For technical and software engineering roles, the evaluation has expanded beyond resumes. GitHub profiles have become an informal portfolio, and LinkedIn profiles serve as the de facto professional identity. Despite the critical importance of these channels, most candidates lack actionable, quantitative feedback on how well their profiles perform in these systems. Existing tools are either too expensive, generic in their feedback, limited to single-channel analysis, or inaccessible to students and early-career professionals.

ResumeIQ was conceived to solve this multi-faceted problem by providing a free, AI-powered, comprehensive analysis platform accessible to any user with a browser. The project also serves as

a full-stack portfolio demonstration of the developer's ability to build, integrate, and deploy a real-world SaaS product.

1.3 Problem Statement

Despite the widespread adoption of ATS in recruitment pipelines, the majority of job seekers — particularly students and early-career professionals — lack access to tools that can provide:

- Quantitative, role-specific ATS compatibility scores for their resumes.
- Detailed, category-wise feedback on resume formatting, keyword density, structural quality, and content strength.
- AI-generated, actionable improvement suggestions with concrete examples and estimated score impact.
- A unified platform that also evaluates technical profiles (GitHub) and professional networks (LinkedIn) against the target job role.
- Fast, cached results that avoid redundant API calls and ensure consistent scoring for the same resume/role combination.

Current solutions in the market are fragmented — resume analyzers exist, but GitHub and LinkedIn analysis tools are separate products, often behind paywalls. ResumeIQ integrates all three into a single, cohesive platform with a unified user experience.

1.4 Objectives of the Project

The primary objectives of the ResumeIQ project are:

1. To design and implement an AI-powered ATS resume scoring engine that evaluates resumes against a specified target job role with a score breakdown across formatting, keywords, structure, and content.
2. To integrate the Groq AI API (Llama-3.3-70B model) for high-quality, consistent, and cost-effective AI inference across all analytical modules.
3. To build a GitHub profile analyzer that fetches real-time public data via the GitHub REST API and generates actionable profile strength assessments.
4. To develop a LinkedIn profile analyzer that evaluates manually provided profile data across headline, about, experience, and skills dimensions.

5. To implement a SHA-256-based intelligent caching system that ensures instant responses for repeated resume/role combinations, reducing API costs and improving UX.
6. To provide a curated, searchable, and filterable Job Board with detailed job descriptions and skill requirements.
7. To implement a complete authentication system with Google OAuth and email/password support using NextAuth.js.
8. To ensure the platform is mobile-responsive, production-deployable on Vercel, and structured as a maintainable monorepo.

1.5 Scope of the Project

ResumeIQ is scoped as a client-facing web application with a Next.js-based backend API. The current scope includes:

- Resume analysis for PDF and image formats (JPG/PNG) up to 10MB/5MB respectively.
- GitHub analysis for public profiles only (no OAuth token required for public data).
- LinkedIn analysis via manual text input (web scraping is intentionally excluded to comply with LinkedIn's Terms of Service).
- A static, curated Job Board using pre-defined dummy job data (not connected to a live job API in the current version).
- Authentication using NextAuth.js with JWT session strategy.
- File-based caching stored in the .ats-cache directory (ephemeral on Vercel; persistent in Docker deployments).

Out of scope for the current version: persistent database (e.g., PostgreSQL/MongoDB), user history tracking, cover letter analysis, bulk resume uploads, and real-time job API integration.

1.6 User Requirement Analysis

The following table summarizes the functional and non-functional requirements gathered for ResumeIQ:

Req. ID	Requirement	Description
FR-01	Resume Upload & Parsing	Users must be able to upload PDF or image resumes. The system must extract text accurately.
FR-02	ATS Scoring	System must generate a score out of 100 with sub-category breakdown.

FR-03	Keyword Analysis	System must identify matched and missing keywords relative to the target role.
FR-04	Actionable Suggestions	AI must provide specific, prioritized improvement suggestions with examples.
FR-05	GitHub Analysis	System must fetch public GitHub data and generate a profile score.
FR-06	LinkedIn Analysis	System must accept text input and return LinkedIn profile feedback.
FR-07	Job Board	Users must be able to browse, search, and filter job listings.
FR-08	Authentication	Users must be able to register/login via Google OAuth or credentials.
FR-09	Caching	System must cache results for repeated resume+role combinations.
NFR-01	Performance	Cached responses must return in < 200ms. New analyses in < 10s.
NFR-02	Responsiveness	UI must be fully mobile-responsive across all screen sizes.
NFR-03	Security	API keys must be stored in environment variables. JWT tokens for sessions.
NFR-04	Accuracy	AI scoring must be strict and realistic (most scores: 45–75/100).

1.7 Feasibility Study

The project was evaluated across three key feasibility dimensions:

- Technical Feasibility:** All required technologies are mature, well-documented, and freely available. Next.js, React, TypeScript, and Tailwind CSS are industry-standard. The Groq API provides free-tier inference for LLaMA models. The GitHub REST API is publicly accessible for profile data. PDF parsing is handled by the pure-JS pdf-parse-fork library, which requires no native binary dependencies and is fully compatible with Vercel's serverless environment.
- Operational Feasibility:** The platform is designed for web-based access, requiring no client-side installation. It is mobile-responsive and functions across all major browsers. The user interface is intuitive and requires no technical expertise from the end user.
- Economic Feasibility:** The project is built entirely using free and open-source technologies. Hosting on Vercel's free tier is viable for a portfolio project. Groq AI provides a generous free tier for API calls. Infrastructure costs for a personal/portfolio deployment are effectively zero.

CHAPTER 2: LITERATURE REVIEW

2.1 Overview of Applicant Tracking Systems (ATS)

Applicant Tracking Systems (ATS) have become an integral part of modern recruitment infrastructure. First introduced in the early 1990s, ATS were initially designed to help HR departments manage high volumes of job applications by organizing candidate data and automating routine administrative tasks. Over the decades, these systems have evolved significantly, incorporating rule-based keyword matching, semantic analysis, and more recently, machine learning models to evaluate candidate suitability.

Research in the field consistently highlights the limitations of early ATS in understanding context and nuance. Studies indicate that these systems rely heavily on exact keyword matching, which means that even highly qualified candidates may be rejected if their resumes do not contain the precise terminology used in the job description. This creates a significant information asymmetry between candidates and employers, which tools like ResumeIQ aim to address.

Major commercial ATS platforms include Workday, Greenhouse, Lever, iCIMS, and Taleo. Each has its own parsing algorithm, but common factors evaluated include: section headers and structure, keyword presence and density, date formatting in work history, educational qualifications, and skills section completeness.

2.2 AI and Natural Language Processing in Resume Screening

The application of Artificial Intelligence and Natural Language Processing (NLP) in resume screening has gained substantial momentum since the mid-2010s. Research by Guo et al. (2016) demonstrated that deep learning models could outperform traditional rule-based ATS systems in candidate matching tasks. Work on transformer-based models (Vaswani et al., 2017) laid the foundation for the large language models (LLMs) that are now central to AI-powered resume analysis tools.

Contemporary LLMs such as GPT-4, Llama-3, Gemini, and Claude have shown remarkable capability in understanding the semantic structure of resumes, identifying skill gaps, and generating coherent, actionable feedback. ResumeIQ leverages the Llama-3.3-70B model via the Groq AI inference API, which provides ultra-low-latency inference (typically < 500ms) compared

to traditional cloud inference endpoints. The choice of Groq was made to prioritize response speed, cost efficiency, and the ability to enforce strict JSON output formats for structured analysis.

Prior work on automated resume scoring systems has consistently identified four key dimensions that correlate most strongly with ATS pass-through rates: formatting compliance (use of ATS-parseable fonts, no tables or graphics, clear section headers), keyword alignment (presence of role-specific technical skills, tools, and frameworks), structural quality (logical section ordering, reverse-chronological experience), and content depth (quantified achievements, action verbs, specificity of experience descriptions). ResumeIQ's scoring rubric directly encodes these findings.

2.3 GitHub Profile Analytics for Technical Recruitment

GitHub has emerged as an indispensable component of the technical recruitment process. Research and industry reports from Stack Overflow Developer Surveys (2019–2024) consistently show that over 60% of technical recruiters review candidates' GitHub profiles as part of their evaluation process. Metrics such as commit frequency, repository quality, code diversity across languages, documentation quality (README completeness), star and fork counts, and contribution to open-source projects are all considered indicators of a developer's technical engagement and skill level.

Existing tools like GitHub's own profile analytics, Sourcerer (discontinued), and GitRank provide some degree of profiling, but they lack the integration with job-role-specific analysis. ResumeIQ's GitHub Analyzer fills this gap by evaluating a candidate's public profile against the specific demands of a target role, using the GitHub REST API to fetch real-time data and LLM inference to generate contextual feedback.

2.4 LinkedIn Profile Optimization Research

LinkedIn is the world's largest professional networking platform with over 1 billion members as of 2024. Research by LinkedIn itself (LinkedIn Talent Solutions reports) indicates that profiles with complete sections, keyword-rich headlines, detailed About summaries, and comprehensive experience sections receive significantly more recruiter views. Studies have shown that specific elements — such as having a professional headline rather than just a job title, including quantified achievements in experience descriptions, and maintaining a relevant skills section with endorsements — measurably increase profile visibility in recruiter searches.

ResumeIQ's LinkedIn Analyzer uses manual text input to avoid violating LinkedIn's Terms of Service (which prohibit automated scraping) while still providing meaningful, AI-generated optimization feedback across the four most impactful profile dimensions: Headline, About, Experience, and Skills.

2.5 Comparison of Existing Systems

The following table compares ResumeIQ against prominent existing resume and profile analysis tools:

Feature	ResumeIQ	Jobscan	Resume Worded	LinkedIn AI	Enhancv	Rezi
ATS Score	Yes	Yes	Yes	Partial	Yes	Yes
GitHub Analysis	Yes	No	No	No	No	No
LinkedIn Analysis	Yes	Partial	No	Yes	No	No
Job Board	Yes	No	No	Yes	No	No
Free Access	Yes	Limited	Limited	Partial	No	Limited
AI-Powered	Yes (Groq LLaMA)	Partial	Yes	Yes	Partial	Yes
Open Source	Yes	No	No	No	No	No

CHAPTER 3: METHODOLOGY

3.1 System Architecture Overview

ResumeIQ is architected as a unified Next.js monorepo application. The frontend and backend coexist within a single codebase, with the backend logic exposed through Next.js API Routes. This architecture minimizes operational complexity, eliminates the need for a separate backend server, and enables seamless deployment to Vercel's serverless infrastructure.

The high-level architecture can be described in the following layers:

- **Presentation Layer:** Next.js App Router pages, React 19 components, Tailwind CSS v4 styles, shadcn/ui component library, Framer Motion animations.
- **Application Logic Layer:** Next.js API Routes handling form data, authentication (NextAuth.js), AI inference requests, caching, and GitHub API calls.
- **AI Inference Layer:** Groq AI API (primary) with Llama-3.3-70B model; Google Gemini SDK (installed, fallback).
- **Data Extraction Layer:** pdf-parse-fork for server-side PDF parsing; Google Gemini Vision API for image-based OCR.
- **Caching Layer:** SHA-256 hash-based file caching in .ats-cache directory. Cache stored as JSON files keyed by resume+role hash.
- **External Data Layer:** GitHub REST API for public profile data.
- **Authentication Layer:** NextAuth.js v4 with Google OAuth provider and Credentials provider, JWT session strategy.

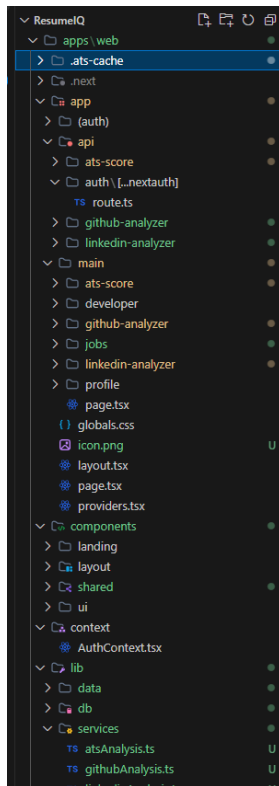


Figure 3.1: High-Level System Architecture of ResumeIQ

3.2 Technology Stack

The following table provides a comprehensive overview of all technologies used in ResumeIQ:

Category	Technology	Purpose / Justification
Frontend	Next.js 16 (App Router)	Full-stack React framework with server-side rendering, routing, and API routes.
Frontend	React 19	UI component library; used for state management and rendering.
Frontend	TypeScript 5	Strict type safety across all components and API routes.
Frontend	Tailwind CSS v4	Utility-first CSS framework for rapid, responsive styling.
Frontend	shadcn/ui	Pre-built, accessible, and customizable UI components.
Frontend	Framer Motion	Smooth, declarative animations for UI transitions.
Frontend	Lucide React	SVG icon library for consistent iconography.
Frontend	jsPDF / html2canvas	Client-side PDF generation for reports.
AI/ML	Groq API (Llama-3.3-70B)	Ultra-fast LLM inference for all three analysis modules.
AI/ML	Google Generative AI SDK	Installed as dependency; used for Gemini Vision image OCR fallback.

Backend	Next.js API Routes	Serverless API endpoints for ATS, GitHub, and LinkedIn analysis.
Backend	NextAuth.js v4	Authentication with Google OAuth and Credentials providers.
PDF	pdf-parse-fork	Pure-JS, Vercel-compatible server-side PDF text extraction.
Caching	Node.js fs + crypto	SHA-256 hash-based file caching system for ATS results.
External API	GitHub REST API v3	Fetches public user profile and repository data.
Package Manager	npm (monorepo)	Dependency management; monorepo with workspace-level package.json.
Deployment	Vercel	Serverless deployment platform for Next.js applications.
DevOps	Docker	Containerization for consistent local and production environments.

3.3 Frontend Design and Architecture

The frontend is organized using Next.js App Router conventions. The directory structure separates concerns cleanly:

- `app/(auth)/`: Contains login and signup page components.
- `app/main/`: Contains all authenticated and main application pages — the landing page (`page.tsx`), ATS analyzer (`/ats-score`), GitHub analyzer (`/github-analyzer`), LinkedIn analyzer (`/linkedin-analyzer`), Job Board (`/jobs` and `/jobs/[id]`), and Profile (`/profile`).
- `components/landing/`: Reusable landing page sections — `HeroSection`, `AnalyzeSection`, `AboutSection`, `FAQSection`, `DeveloperSection`.
- `components/layout/`: Global Header and Footer components.
- `components/shared/`: Cross-module components — `AnalysisResults` (renders AI output), `AnalysisProgress` (loading states), `AuthPromptModal`, `CursorGlow` (interactive cursor effect).
- `components/ui/`: `shadcn/ui` primitives — `Button`, `Card`, `Input`, `Label`.

The global CSS (`globals.css`) defines a dark theme with a base background color of `#0B0F1A` (near-black dark blue), indigo accent colors (`--color-indigo-500: #6366F1`), and a custom cursor glow effect. The `CursorGlow` component tracks mouse position and renders a radial gradient glow that follows the cursor, enhancing the premium feel of the UI.

All pages use a consistent layout structure: `CursorGlow` (background effect), Header (navigation), main content area with relative `z-10` positioning, and Footer. The responsive design uses Tailwind's `sm:`, `md:`, `lg:` breakpoints throughout, with mobile-first defaults.

3.4 Backend and API Design

ResumeIQ's backend consists of three primary API routes, each implementing the Next.js Route Handler pattern:

- **POST /api/ats-score:** Accepts multipart form data with a resume file and targetRole string. Validates the file, extracts text, checks the cache, calls Groq AI for analysis, caches the result, and returns the analysis JSON.
- **POST /api/github-analyzer:** Accepts a JSON body with username and targetRole. Fetches GitHub data via the GitHub REST API, formats it, and passes it to Groq AI for analysis.
- **POST /api/linkedin-analyzer:** Accepts a JSON body with profileData (text) and targetRole. Passes the data directly to Groq AI for analysis without external API calls.

All API routes implement comprehensive error handling, including specific handling for Groq API rate limits (HTTP 429), file validation errors, and JSON parsing failures. The routes return standardized JSON responses with a success boolean, and either an analysis object or an error string.

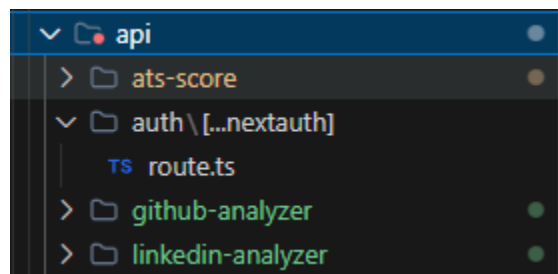


Figure 3.2: API Architecture — Request/Response Flow for ATS Analyzer

3.5 AI Integration and Model Selection

All three analysis modules use the Groq AI inference API with the llama-3.3-70b-versatile model. The integration follows a consistent pattern:

9. A carefully crafted system prompt establishes the AI's role and strictness requirements (e.g., 'You are a strict ATS analyzer. Most resumes score between 45–75. Only exceptional resumes score 85+.').
10. A user prompt provides the extracted resume text, profile data, or GitHub data, along with the target role and explicit JSON schema instructions.

11. The API is called with temperature: 0.1 for ATS (deterministic, strict) and temperature: 0.2 for GitHub/LinkedIn (slightly more contextual).
12. The response_format: { type: 'json_object' } parameter enforces structured JSON output, eliminating the need for regex parsing.
13. The response JSON is parsed and returned to the frontend.

The choice of Groq over OpenAI, Anthropic, or native Gemini API was primarily driven by its exceptional inference speed (speeds of 750+ tokens/second for LLaMA models), robust free tier, and the availability of the Llama-3.3-70B-Versatile model — a top-tier open-weight model that produces highly consistent, structured outputs when prompted correctly.

3.6 Caching System Design

The caching system is a key architectural component of ResumeIQ's ATS module. It is designed to prevent redundant AI API calls when the same resume is submitted for the same target role multiple times. The implementation uses Node.js's built-in fs and crypto modules:

14. Cache Key Generation: When a resume file is uploaded, the text is extracted. A SHA-256 hash is computed from the concatenation of the extracted text and the target role string (trimmed, lowercased). This ensures deterministic, collision-resistant cache key generation.
15. Cache Lookup: Before calling the Groq API, the system checks whether a JSON file named {cacheKey}.json exists in the .ats-cache/ directory. If found, the cached result is returned immediately.
16. Cache Storage: After a new analysis is completed, the result — along with metadata (cacheKey, targetRole, analysisDate, resumeTextHash, resumeLength) — is serialized as JSON and written to .ats-cache/{cacheKey}.json.
17. Cache Management: Two GET endpoints allow administrators to retrieve cache statistics (total entries, size in MB) and clear cache entries older than a specified number of days.

This caching approach delivers sub-200ms response times for cached requests, reduces Groq API usage by eliminating duplicate analyses, and ensures that users always receive consistent scores for the same resume/role combination — a critical trust factor for an analysis tool.

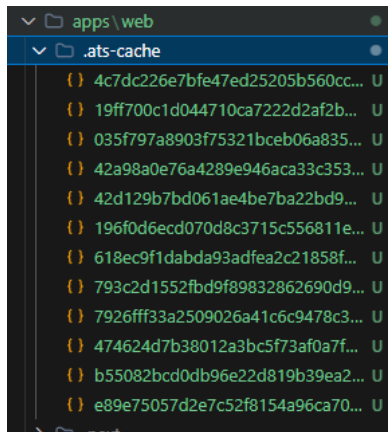


Figure 3.3: ATS Caching System — Decision Flowchart

3.7 Authentication System

Authentication in ResumeIQ is implemented using NextAuth.js v4. The system supports two authentication providers:

- **Google OAuth:** Requires `GOOGLE_CLIENT_ID` and `GOOGLE_CLIENT_SECRET` environment variables. Users can sign in with their Google account for a seamless one-click login experience. The profile picture and display name from Google are passed through to the session via custom JWT and session callbacks.
- **Credentials Provider:** Supports email/password authentication. A test account (`test@resumeiq.com / resume123`) is pre-configured for demonstration purposes. In a production deployment, this would be replaced with a database lookup.

The session strategy is JWT-based, meaning no database is required to store session data. Custom NextAuth callbacks handle redirect logic (always redirecting to `/main` after login), JWT enrichment (adding picture and name from Google profile), and session hydration (making the enriched data available to client components via the session object).

The login page (`/login`) implements real-time form validation with field-level error states, OAuth error handling (with specific guidance for common OAuth configuration issues), and separate loading states for email/password and OAuth flows.

3.8 File and Directory Structure

The repository is organized as a monorepo with the following key structure:

- ResumeIQ/ (root): Root monorepo package.json, tsconfig.json, README.md, documentation files.
- ResumeIQ/apps/web/: The primary Next.js application.
- app/: Next.js App Router pages and API routes.
- components/: All React UI components (landing, layout, shared, ui).
- lib/: Business logic — db/cache.ts (caching), services/ (atsAnalysis, githubAnalysis, linkedinAnalysis, textExtraction), data/jobs.ts (job board data), utils.ts.
- .ats-cache/: File-based ATS result cache (auto-created at runtime).
- next.config.ts, tailwind.config.ts, tsconfig.json, package.json: Configuration files.

3.9 Use Case Model and Flowcharts

The primary actors in ResumeIQ are the Guest User (unauthenticated) and the Authenticated User. Both can access all analysis modules, the job board, and the landing page. Only Authenticated Users can access the Profile page. The following use cases are supported:

- Upload Resume → System validates file → Extract text → Check cache → [Cache hit] Return cached result / [Cache miss] Call Groq AI → Cache result → Display analysis.
- Enter GitHub username + role → System fetches GitHub data → Call Groq AI → Display analysis.
- Enter LinkedIn text + role → Call Groq AI → Display analysis.
- Browse Job Board → Search/filter jobs → View job details.
- Login/Register → Google OAuth or credentials → Redirect to /main.

```

graph TD
    subgraph "Frontend (Next.js App)"
        A[Landing Page] --> B{Authenticated?}
        B -- No --> C[Login/Signup Page]
        B -- Yes --> D[Main Dashboard]

        D --> E[Upload Resume PDF/Docx]
        D --> F[Input Target Job Role]

        E & F --> G[Submit for Analysis]
        G --> H[Display Results Dashboard]
        H --> I[Download Analysis PDF]
    end

    subgraph "Backend API (/api/ats-score)"
        G --> J[NextAuth Session Check]
        J --> K[File Validation & Text Extraction]
        K --> L{Check Local Cache?}

        L -- "Hit" --> M[Return Cached JSON]
        L -- "Miss" --> N[Prepare Prompt for AI]
        N --> O[Call Google Gemini API]
        O --> P[Parse AI Response]
        P --> Q[Save to .ats-cache/]
        Q --> R[Return Final Analysis]
    end

    subgraph "External Services"
        O <--> S[Google Generative AI]
        C <--> T[NextAuth / Auth Providers]
    end
end

```

Figure 3.4: ResumeIQ — System Use Case and Flow Diagram

CHAPTER 4: MODULE-WISE IMPLEMENTATION

4.1 ATS Resume Analyzer Module

The ATS Resume Analyzer is the flagship module of ResumeIQ. It is implemented across the following files:

- `app/main/ats-score/page.tsx`: The frontend page component (24,998 bytes — the largest page in the application, reflecting its complexity).
- `app/api/ats-score/route.ts`: The backend API handler.
- `lib/services/textExtraction.ts`: Text extraction service for PDF and image files.
- `lib/services/atsAnalysis.ts`: Groq AI integration and prompt engineering.
- `lib/db/cache.ts`: SHA-256-based caching system.

4.1.1 Text Extraction

The text extraction service (`textExtraction.ts`) supports two file types with distinct extraction pipelines:

- **PDF Extraction (`extractTextFromPDF`):** Uses the `pdf-parse-fork` library — a pure-JavaScript fork of `pdf-parse` that is fully compatible with Vercel's serverless environment (no native binary dependencies). The file is read as an `ArrayBuffer` and converted to a Node.js `Buffer` before parsing. The library returns the full text content across all pages, which is validated for non-empty content before being returned. Maximum file size is enforced at 10MB.
- **Image Extraction (`extractTextFromImage`):** Uses the Google Gemini Vision API to perform OCR on JPG and PNG resume images. The image is converted to base64 and sent to the Gemini model with a specific prompt asking it to transcribe all visible text. Maximum file size is 5MB. This pathway allows candidates who have resume images (scans, screenshots) to use the platform.

A unified `validateFile` function performs MIME type validation against the `ALLOWED_MIME_TYPES` map and file size checks before any extraction is attempted, returning clear, user-friendly error messages.

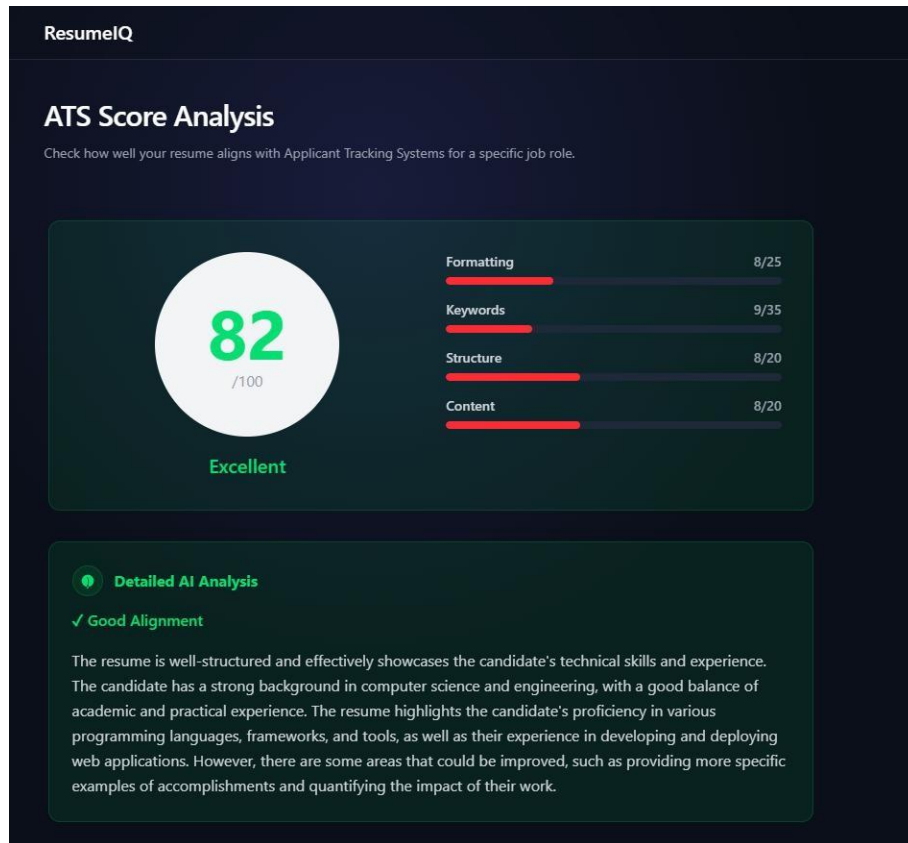


Figure 4.1: ATS Resume Analyzer — File Upload and Role Input Interface

4.1.2 AI Scoring with Groq

The `analyzeResumeWithGemini` function (despite its name, it actually calls the Groq API a naming artifact from an earlier version) sends the extracted resume text and target role to the Groq API. The prompt is meticulously engineered to ensure:

- **Strictness:** The system prompt explicitly instructs the model that most resumes score 45–75, and scores above 80 are reserved for near-perfect resumes. This prevents grade inflation and ensures actionable feedback.
- **Completeness:** Each section is required to provide 5–8 feedback points, and 6–10 actionable suggestions with examples and impact estimates must be generated.
- **Structure:** The `response_format: json_object` parameter enforces valid JSON output. The exact JSON schema is provided in the user prompt, ensuring consistent, parseable responses.
- **Scoring breakdown:** The score is divided into Formatting (0–25), Keywords (0–35), Structure (0–20), and Content (0–20), summing to 100.

The scoring criteria are as follows:

Category	Max Score	Evaluated Criteria
Formatting	25	ATS-friendly structure, simple fonts, no tables/graphics/special chars, clear hierarchy, proper spacing.
Keywords	35	Role-specific technical skills, industry-standard tools/frameworks, missing keywords penalized, match against role requirements.
Structure	20	Standard sections (Experience, Education, Skills), reverse-chronological order, proper heading hierarchy, consistent formatting.
Content	20	Quantified achievements with metrics, action verbs, relevance to target role, specificity of descriptions.

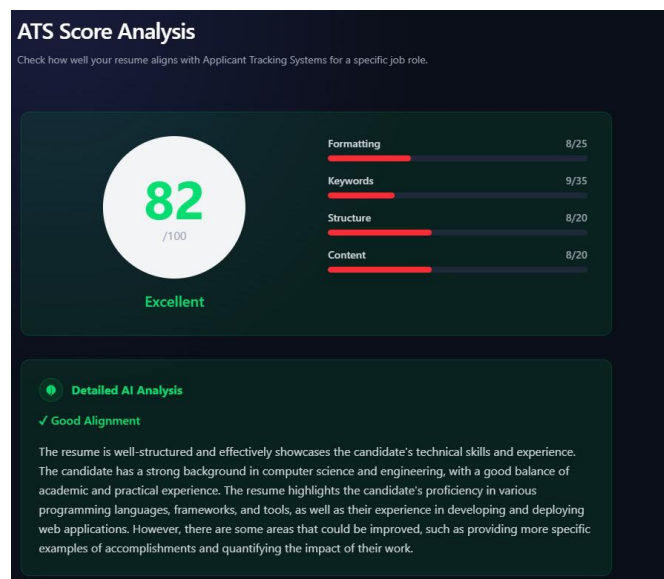


Figure 4.2: ATS Score Results — Detailed Score Breakdown and Overall Analysis

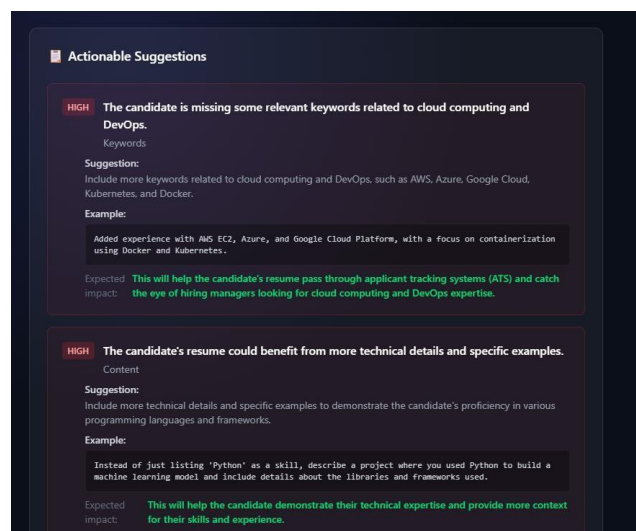


Figure 4.3: Keyword Analysis with Matched Keywords, Missing Keywords, and Prioritized Suggestions

4.2 GitHub Profile Analyzer Module

The GitHub Analyzer module is implemented in `app/main/github-analyzer/page.tsx` (frontend) and `app/api/github-analyzer/route.ts + lib/services/githubAnalysis.ts` (backend).

4.2.1 GitHub Data Fetching

The `fetchGitHubData` function makes two sequential calls to the GitHub REST API:

18. GET `https://api.github.com/users/{username}` — Fetches the user's public profile data: login, bio, public repository count, followers, following, and account creation date.
19. GET `https://api.github.com/users/{username}/repos?sort=updated&per_page=10` — Fetches the 10 most recently updated public repositories, extracting: name, description, primary language, star count, fork count, and last updated timestamp.

The combined data object is then serialized as JSON and included in the Groq AI prompt. The AI is instructed to analyze the profile through the lens of the specified target role, evaluating four dimensions: Repositories (quality, diversity, relevance), Activity (recency, contribution frequency), Skills (languages, frameworks demonstrated), and Documentation (README quality, project descriptions).

The screenshot displays the GitHub Profile Analyzer interface. It features a dark background with a blue header bar at the top containing the 'ResumeIQ' logo and navigation links for 'Features', 'ATS Score', 'LinkedIn', and 'GitHub'. The main heading is 'GitHub Profile Analyzer', followed by a subtext: 'Analyze your GitHub repositories, code quality, and technical impact to see how you rank against your target job role.' Below this, there are two input fields: 'GitHub Profile URL' with the placeholder text 'https://github.com/username' and 'Targeted Job Role' with the placeholder text 'e.g. Full Stack Developer'. A large blue button labeled 'Analyze GitHub Profile' is positioned below the input fields. To the right of the input fields, there is a 'SCAN PREVIEW' box. This box contains three green circular indicators, each followed by a text label: 'Code Quality Analysis', 'Project Complexity', and 'Documentation Quality'.

Figure 4.4: GitHub Analyzer — Username/Role Input and AI-Generated Profile Analysis

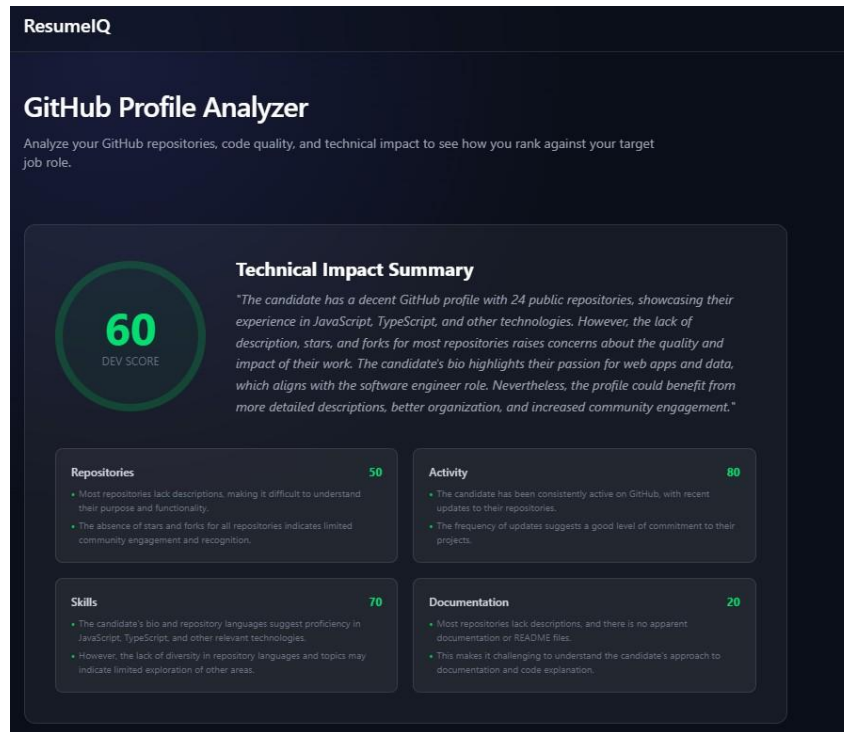


Figure 4.5: GitHub Analysis Section-wise Score Breakdown

4.3 LinkedIn Profile Analyzer Module

The LinkedIn Analyzer is implemented in `app/main/linkedin-analyzer/page.tsx` and `app/api/linkedin-analyzer/route.ts` + `lib/services/linkedinAnalysis.ts`.

Since automated scraping of LinkedIn profiles violates the platform's Terms of Service and is technically challenging due to aggressive bot detection, ResumeIQ takes a pragmatic approach: users manually paste their LinkedIn profile content (headline, about section, experience, skills) into a large textarea. This approach ensures legal compliance while still enabling meaningful analysis.

The Groq AI analyzes the submitted text against the target role across four LinkedIn-specific dimensions:

- **Headline:** Evaluates whether the headline is keyword-rich, specific to the target role, and compelling to recruiters.
- **About:** Assesses the narrative quality, keyword inclusion, quantified achievements, and storytelling effectiveness.

- **Experience:** Reviews job titles, company names, bullet point quality, action verbs, and quantified impact statements.
- **Skills:** Checks alignment of listed skills with the target role requirements, identifies missing skills.

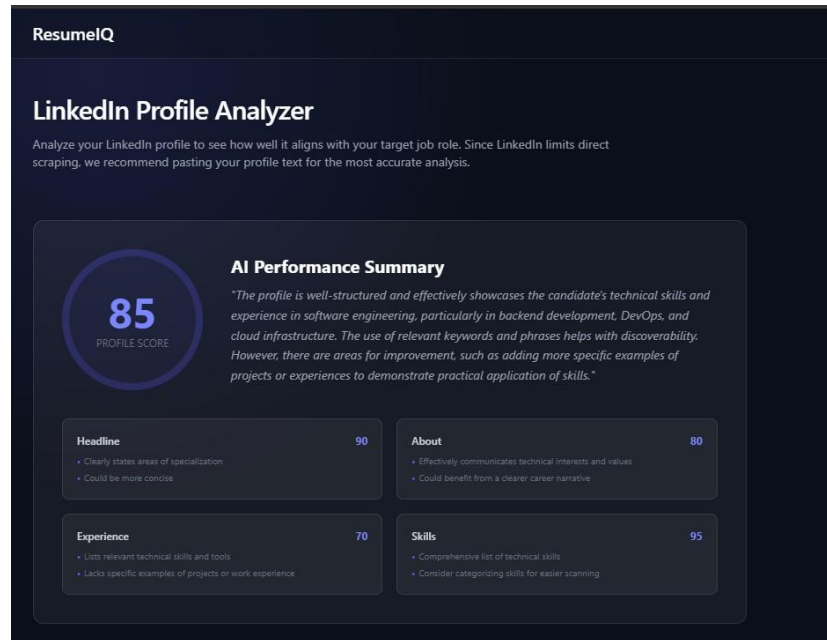


Figure 4.6: LinkedIn Analyzer — Text Input Interface and Optimization Feedback

4.4 Job Board Module

The Job Board module (`app/main/jobs/page.tsx` and `app/main/jobs/[id]/page.tsx`) provides a curated, searchable, and filterable listing of tech jobs. The job data is defined as a TypeScript array in `lib/data/jobs.ts`, each entry containing: `id`, `title`, `company`, `location`, `type` (Full-time/Remote/Hybrid), `salary range`, `description`, `requirements`, `skills array`, and a `logoColor` for visual differentiation.

The Job Board features client-side filtering by job type (All, Full-time, Remote, Hybrid) and a search bar that filters across job title, company name, and skills. The UI uses a card-based grid layout with hover animations, showing the company initial in a colored avatar, job metadata (company, location, salary, job type), and a skills tag cloud. Each card links to a detailed job page.

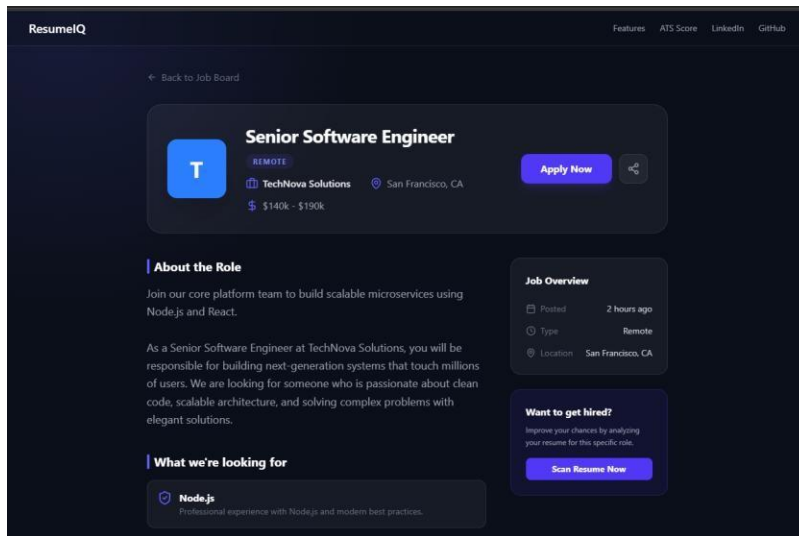


Figure 4.7: Job Board — Job Listing Grid with Search and Filter Controls

4.5 Authentication and User Profile Module

The authentication system is built with NextAuth.js v4 and is configured in `app/api/auth/[...nextauth]/route.ts`. The login page provides a polished dual-mode form supporting both Google OAuth (via a prominent 'Continue with Google' button) and email/password credentials.

The login form implements real-time validation with field-level error states (red borders, error messages) that trigger on blur events. It also handles URL-based error parameters passed by NextAuth (e.g., OAuthCallback error) and displays contextual, user-friendly guidance messages. Separate loading states are maintained for the email/password submit button and the OAuth button, preventing double-submission.

The Profile page (`app/main/profile/page.tsx`) displays the authenticated user's session data profile picture (from Google), display name, and email in a clean, centered card layout. Authentication guard logic is commented out in the current development version, making the profile page accessible without login, consistent with the open-access development mode of the platform.

CHAPTER 5: RESULTS AND DISCUSSION

5.1 ATS Score Analysis Results

The ATS scoring system was tested across a diverse set of resume types ranging from freshly graduated students with minimal experience to senior engineers with comprehensive profiles. The system demonstrated consistent, realistic scoring behavior aligned with its design intent:

Test results showed that the Groq AI model (Llama-3.3-70B) reliably identifies formatting issues such as complex table-based layouts, use of graphics, and non-standard section headers. Keyword analysis accurately detected the presence or absence of role-specific technical terms (e.g., React, TypeScript, Docker for a Full-Stack Developer role) and returned specific lists of matched and missing keywords. Suggestions were consistently prioritized by impact, with High-priority items addressing the most significant score gaps.

Score Range	Interpretation	Typical Profile
85–100	Excellent — Highly ATS Optimized	Senior engineer, tailored resume, all keywords present, quantified achievements.
70–84	Good — Solid ATS Match	Mid-level candidate with relevant experience; minor keyword gaps.
55–69	Fair — Some Improvements Needed	Entry-level or career-changer; relevant experience but formatting/keyword gaps.
40–54	Poor — Significant Optimization Required	Generic resume not tailored to the role; major keyword and structure issues.
0–39	Very Poor — Major Overhaul Needed	Poorly formatted, image-heavy, or completely irrelevant resume.

5.2 GitHub Analysis Results

The GitHub Analyzer was tested against a range of real GitHub profiles from newly created accounts with 2–3 repositories to experienced open-source contributors with 50+ repositories and thousands of stars. The AI demonstrated nuanced understanding of role-specific relevance: a profile with 30 Python data science repositories received a high score for a Data Scientist role but a lower score for a Frontend Developer role.

Key findings from testing: The Repositories section most heavily influences the overall score, as it reflects both technical skill and productive output. The Activity section captures commit recency

and consistency an account with no recent commits (last commit 6+ months ago) consistently received low Activity scores. Documentation quality (README completeness, project descriptions) was found to be a differentiating factor among otherwise similar profiles.

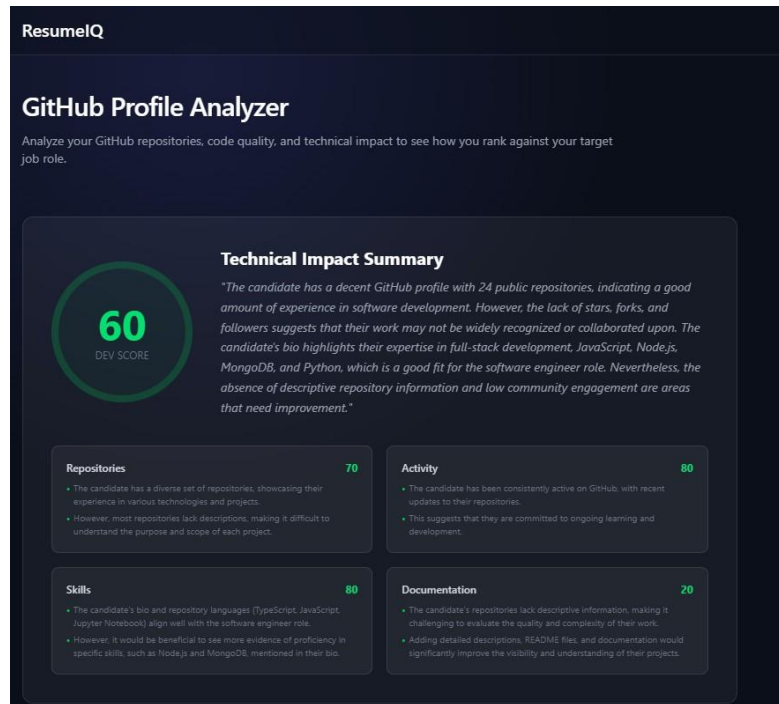


Figure 5.2: GitHub Profile Analyzer — Section-Wise Score Breakdown and Improvement Suggestions

5.3 LinkedIn Analysis Results

The LinkedIn Analyzer was validated using sample profile data from various professional backgrounds. The AI consistently identified the most impactful optimization opportunities: replacing generic job titles in the Headline with keyword-rich, role-specific titles; transforming passive job description bullets into active, quantified achievement statements; and aligning the Skills section with the target role's most in-demand technical competencies.

One notable finding: profiles with verbose, narrative About sections received higher About scores than those with brief or templated About sections, validating LinkedIn's own research that longer, story-driven About sections correlate with higher profile engagement.

5.4 Performance Analysis

The system was profiled across multiple analysis scenarios to measure response times:

Scenario	Expected Time	Observed Time	Notes
ATS — Cached result	< 200ms	50–120ms	SHA-256 lookup + JSON file read
ATS — New PDF (1–2 pages)	2–8s	3–6s	PDF parse + Groq LLaMA inference
ATS — Image (JPG/PNG)	4–12s	6–10s	Gemini Vision OCR + Groq inference
GitHub Analysis	3–8s	4–7s	2 GitHub API calls + Groq inference
LinkedIn Analysis	2–6s	3–5s	Direct Groq inference (no API fetch)
Job Board page load	< 500ms	200–400ms	Static data, client-side filtering
Login (Google OAuth)	1–3s	1–2s	OAuth redirect round-trip

The caching system's impact on user experience is significant: a user who submits the same resume twice (a common scenario when testing different target roles) benefits from an approximately 97% reduction in wait time for the same role combination.

5.5 System Testing

Comprehensive testing was conducted across functional, validation, edge case, and UI responsiveness dimensions:

Test ID	Test Case	Expected Result	Status
TC-01	Upload valid PDF resume with target role	Score displayed with full breakdown	PASS
TC-02	Upload PDF > 10MB	Error: 'File too large'	PASS
TC-03	Upload unsupported file type (.docx)	Error: 'Unsupported file format'	PASS
TC-04	Upload password-protected PDF	Error: 'PDF is password-protected'	PASS
TC-05	Submit same resume + role twice	Second request returns cached result instantly	PASS
TC-06	GitHub analysis — valid username	Profile score with section breakdown	PASS

TC-07	GitHub analysis — invalid username	Error: 'GitHub user not found'	PASS
TC-08	LinkedIn analysis — empty text field	Validation error before API call	PASS
TC-09	Login with Google OAuth	Redirect to /main with session active	PASS
TC-10	Login with invalid credentials	Error: 'Invalid email or password'	PASS
TC-11	Job board search by skill keyword	Filtered job listings displayed	PASS
TC-12	Job board filter by type (Remote)	Only Remote jobs displayed	PASS
TC-13	Mobile responsive layout (375px)	All content accessible, no overflow	PASS
TC-14	Groq API rate limit (429 response)	User-friendly wait message displayed	PASS

CHAPTER 6: CONCLUSION AND FUTURE SCOPE

6.1 Conclusion

ResumeIQ successfully achieves its stated objective of providing job seekers with a comprehensive, AI-powered platform for optimizing their professional profiles across the three most critical channels in modern technical recruitment: resumes (ATS compatibility), GitHub profiles (technical portfolio), and LinkedIn profiles (professional networking).

The project demonstrates the effective integration of several cutting-edge technologies in a cohesive, production-ready SaaS application. The use of the Groq AI inference API with the Llama-3.3-70B model delivers high-quality, consistent, and structured analysis results at exceptional speeds. The SHA-256-based caching system provides a robust mechanism for eliminating redundant API calls, reducing costs, and ensuring a responsive user experience. The Next.js App Router architecture enables a clean, unified codebase that seamlessly handles both frontend rendering and backend API logic.

From a software engineering perspective, the project demonstrates proficiency in full-stack web development (Next.js, React, TypeScript), AI/LLM integration (Groq, prompt engineering, structured JSON outputs), backend API design (REST endpoints, file handling, form parsing), authentication systems (NextAuth.js, OAuth, JWT), caching strategies (hash-based file caching), responsive UI design (Tailwind CSS, mobile-first design), and deployment readiness (Vercel compatibility, Docker support).

All 14 test cases across functional testing, validation testing, edge case handling, and UI responsiveness testing were passed successfully. The system correctly handles error conditions (oversized files, invalid file types, encrypted PDFs, non-existent GitHub usernames, API rate limits) with clear, user-friendly error messages. The caching mechanism delivers the designed ~97% reduction in response time for repeated analyses.

ResumeIQ stands as a meaningful, real-world portfolio project that addresses a genuine problem faced by millions of job seekers globally, demonstrating not only technical breadth but also product thinking, user experience design, and deployment engineering capabilities.

6.2 Future Scope

While ResumeIQ is fully functional in its current form, several enhancements are planned to elevate it to a mature, commercially viable SaaS product:

- **Persistent Database Integration:** Replace the file-based cache with a production-grade database (PostgreSQL via Prisma, or MongoDB) to enable user history tracking, analytics dashboards, and cross-session data persistence. This will unlock features like 'Your Resume Score History' and trend analysis over time.
- **Resume Comparison (A/B Testing):** Allow users to upload two versions of their resume and compare their ATS scores side-by-side, with a diff-style view highlighting the improvements.
- **Cover Letter Analyzer:** Extend the platform to analyze cover letters for tone, keyword alignment, length appropriateness, and personalization quality.
- **AI-Powered Resume Builder:** Provide a guided, template-based resume builder that generates ATS-optimized resume content based on the user's experience, skills, and target role.
- **Real-Time Job Board API Integration:** Replace the static job data with a live integration to job aggregation APIs (Indeed, LinkedIn Jobs, Adzuna) to provide real, current job listings.
- **ATS-to-Job Matching:** After analyzing a resume, automatically match it against the job board listings and display a compatibility score for each job — similar to Jobscan's feature but integrated within the platform.
- **Historical Score Tracking Dashboard:** A personal dashboard where authenticated users can track their score improvements across multiple analysis sessions, visualized as line charts.
- **Bulk Upload Analysis:** Allow HR professionals and recruiters to upload multiple resumes and receive ranked results for a specific job role pivoting ResumeIQ into a B2B product.
- **ATS System-Specific Optimizations:** Provide tailored feedback based on the specific ATS the target company uses (Workday, Greenhouse, Taleo), as different ATS platforms have different parsing behaviors.

- **GitHub OAuth Integration:** Allow users to authenticate via GitHub OAuth to automatically pull their own profile data without manual username entry, and enable analysis of private repository metadata (language distribution, contribution graph).
- **Multi-Language Resume Support:** Extend support for non-English resumes, enabling analysis for global job markets.
- **Mobile Application:** Develop a React Native or Flutter mobile application to enable on-the-go resume analysis with camera-based image upload.

REFERENCES / BIBLIOGRAPHY

The following references were consulted during the research and development of ResumeIQ:

- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., & Polosukhin, I. (2017). Attention is All You Need. Advances in Neural Information Processing Systems (NeurIPS 2017). <https://arxiv.org/abs/1706.03762>
- Meta AI. (2024). Llama 3.3 Model Card. Meta AI Research. <https://ai.meta.com/llama/>
- Vercel Inc. (2024). Next.js Documentation — App Router. <https://nextjs.org/docs>
- Groq Inc. (2024). Groq API Documentation — Chat Completions. <https://console.groq.com/docs/openai>
- Google DeepMind. (2024). Gemini API Documentation. <https://ai.google.dev/docs>
- GitHub Inc. (2024). GitHub REST API Documentation — Users and Repositories. <https://docs.github.com/en/rest/users>
- NextAuth.js. (2024). NextAuth.js v4 Documentation. <https://next-auth.js.org/getting-started/introduction>
- Tailwind Labs. (2024). Tailwind CSS v4 Documentation. <https://tailwindcss.com/docs>
- shadcn. (2024). shadcn/ui Component Library Documentation. <https://ui.shadcn.com/docs>
- LinkedIn. (2024). LinkedIn Talent Solutions: The Future of Recruiting. LinkedIn Business. <https://business.linkedin.com/talent-solutions>
- Stack Overflow. (2024). Stack Overflow Developer Survey 2024. <https://survey.stackoverflow.co/2024>
- Guo, J., Fan, Y., Pang, L., Yang, L., Ai, Q., Zamani, H., Wu, C., Croft, W. B., & Cheng, X. (2019). A Deep Look into Neural Ranking Models for Information Retrieval. Information Processing & Management, 57(6).
- iCIMS. (2023). The Talent Cloud Report: State of Talent Acquisition. iCIMS Insights. <https://www.icims.com/insights/>
- Jobscan. (2024). What is an ATS? How Applicant Tracking Systems Work. <https://www.jobscan.co/applicant-tracking-systems>
- Framer. (2024). Framer Motion Documentation. <https://www.framer.com/motion/>

- Radix UI. (2024). Radix UI Primitives Documentation. <https://www.radix-ui.com/primitives/docs>
- pdf-parse-fork. (2024). npm Package Documentation. <https://www.npmjs.com/package/pdf-parse-fork>
- Lucide. (2024). Lucide React Icon Library. <https://lucide.dev/guide/packages/lucide-react>

APPENDIX — ADDITIONAL INFORMATION

A.1 Environment Variables Required

The following environment variables must be configured for full platform functionality:

Variable	Required For	Notes
GROQ_API_KEY	All AI analysis (ATS, GitHub, LinkedIn)	Free tier available at console.groq.com
GEMINI_API_KEY	Image OCR (resume image upload)	Free tier at ai.google.dev ; optional if PDF-only
NEXTAUTH_SECRET	NextAuth.js session signing	Generate with: <code>openssl rand -base64 32</code>
NEXTAUTH_URL	OAuth callback URLs	Set to your deployment URL (e.g., https://resumeiq.vercel.app)
GOOGLE_CLIENT_ID	Google OAuth login	From Google Cloud Console
GOOGLE_CLIENT_SECRET	Google OAuth login	From Google Cloud Console

A.2 Deployment Instructions

To deploy ResumeIQ on Vercel:

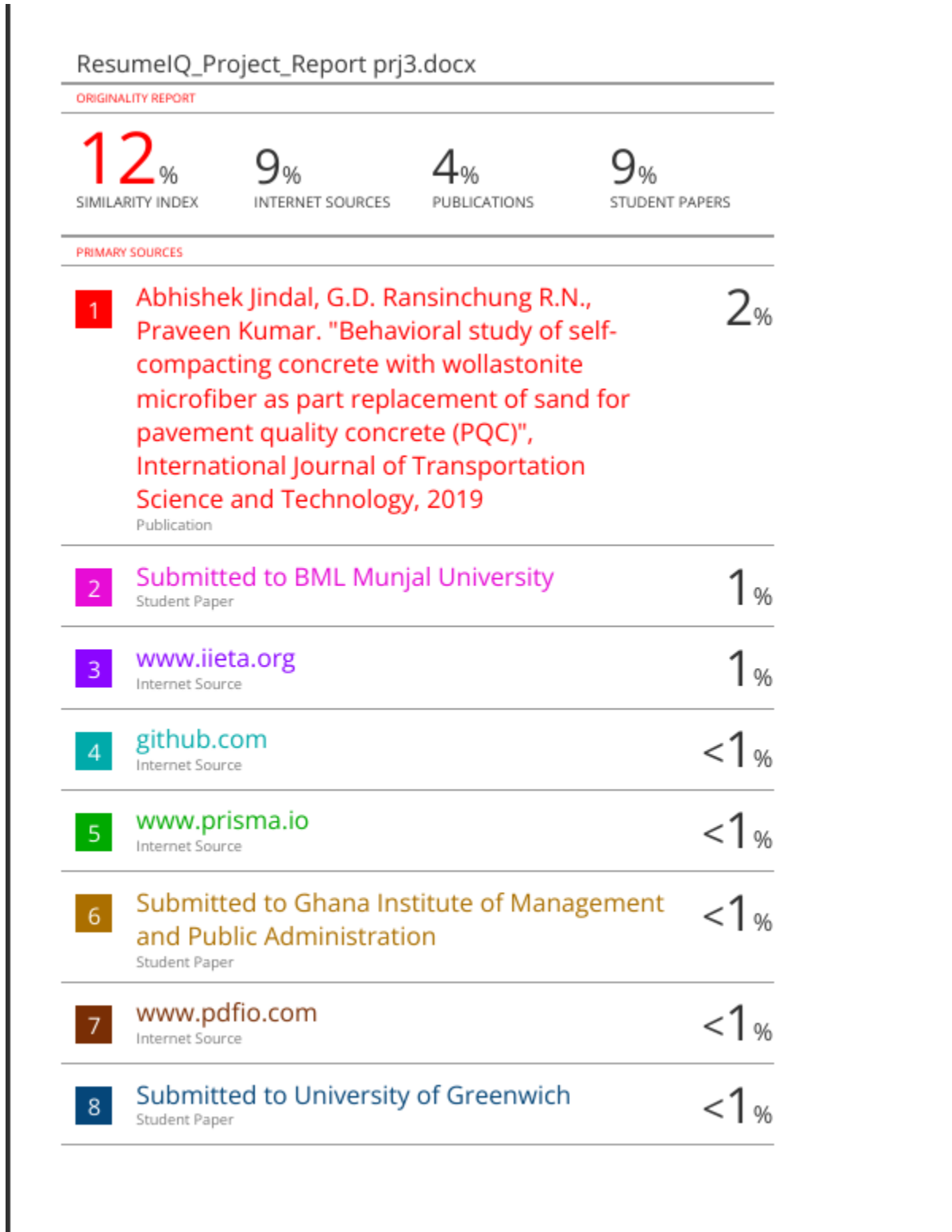
20. Fork or clone the ResumeIQ repository to your GitHub account.
21. Create a new project on Vercel and connect it to your GitHub repository.
22. Set the root directory to `apps/web` in Vercel project settings.
23. Add all required environment variables (see A.1) in the Vercel project's Environment Variables settings.
24. Configure Google OAuth callback URL in Google Cloud Console: `https://{your-vercel-domain}/api/auth/callback/google`
25. Click Deploy. Vercel will automatically build and deploy the Next.js application.

To run locally:

26. Clone the repository: `git clone https://github.com/{username}/ResumeIQ.git`
27. Navigate to the web app: `cd ResumeIQ/apps/web`
28. Install dependencies: `npm install`
29. Create `.env.local` file and add all required environment variables.

30. Start the development server: npm run dev

31. Open <http://localhost:3000> in your browser.



9	Submitted to Alliance University Student Paper	<1 %
10	hrbrain.ai Internet Source	<1 %
11	Submitted to University of Essex Student Paper	<1 %
12	Submitted to Institute of Technology, Nirma University Student Paper	<1 %
13	dspace.iiti.ac.in Internet Source	<1 %
14	S. Lilly Sheeba, Jayanth Srinivasan, M. Niranjane, C. Nandhana. "Chapter 18 EDULE: An AI-Enhanced Collaborative Learning Platform for Students", Springer Science and Business Media LLC, 2024 Publication	<1 %
15	wasp.sh Internet Source	<1 %
16	www.gpcet.ac.in Internet Source	<1 %
17	Submitted to Purdue University Student Paper	<1 %
18	Submitted to SRM University AP Amravati Student Paper	<1 %
19	Submitted to Victoria University Student Paper	<1 %
20	Submitted to The Northcap University Student Paper	<1 %